Квантизация весов LLM и инференс (Triton)

Отчёт по проекту efmml: эксперимент 1, затем план из `ORIGINAL_TASK.md` и расписание на 4 недели.

1. Эксперимент 1

Реализованы ядра в `src/kernels/triton/int4_ops.py` (квантизация + matmul W4) и `matmul_w16_triton.py` (matmul W16 на Triton для сравнения). Цель по памяти: весы в int4 занимают ~в 4 раза меньше, чем fp16 (см. `fp16_weight_bytes` / `int4_packed_weight_bytes`).

Корректность: `pytest tests/`.

1.1 Откуда берутся строки таблицы

Каждая **строка** — это один замер для пары *(слой, размер батча токенов M)*.

1. Список слоёв и размеров весов **(N, K)** задаётся в `src/llama_1b_shapes.py`: константа `LLAMA_1B_LINEAR_SHAPES` (имя проекции `layer` и выходная/входная размерность в терминах `nn.Linear`: вес `(N, K)`, прямой проход `y = x @ W.T`, то есть активация `(M, K)`).
2. Размеры **M** перебираются из `TOKEN_BATCH_SIZES = (128, 512, 2048)` — как в задании для матрицы активаций X.
3. В `experiments/exp01_bench_matmul.py` двойной цикл по слоям и по `M`: для каждой пары создаётся случайная `x` формы `(M, K)` в `bfloat16` на GPU, затем вызывается `bench()` (прогрев `warmup`, усреднение по `iters`, синхронизация CUDA).
4. Результаты пишутся в JSON **в порядке вложенных циклов** (все M для первого слоя, затем следующий слой).

Перед замером — **трёхфазный прогрев** по всем слоям и всем M (отдельно W16 Triton, отдельно `torch.matmul`, отдельно W4 Triton), чтобы снизить риск сбоев JIT на Python 3.13.

1.2 Что означают колонки таблицы

Колонка	Смысл
layer	Имя линейного слоя из `LLAMA_1B_LINEAR_SHAPES` (например `q_proj`, `gate_proj`).
M	Число строк матрицы активаций `(M, K)` (токены 128 / 512 / 2048).
N	`out_features` слоя (строка веса W).
K	`in_features` (вход для `x @ W.T`).
W16 Triton (ms)	Среднее время `matmul_x_bf16_w16_triton(x, W_bf16)`.
W16 torch (ms)	Среднее время `torch.matmul(x, W_bf16.t())` (обычно cuBLAS).
W4 Triton (ms)	Среднее время `matmul_x_bf16_w4(x, packed, scales)` после квантизации веса в int4.
W4÷W16tr	`(время W4) / (время W16 Triton)`; в JSON — `w16_triton_over_w4`. Если больше 1, W4 медленнее W16 Triton на этом замере.

1.3 Результаты замеров

Параметры замера: `warmup=5`, `iters=20` (см. `bench()` в `experiments/exp01_bench_matmul.py`). При сборке этого отчёта, если в JSON не было `ms_w16_triton`, значения дописываются тем же ядром и прогревом (`measure_ms_w16_triton_grid` в том же файле).

layer	M	N	K	W16 Triton (ms)	W16 torch (ms)	W4 Triton (ms)	W4÷W16tr
q_proj	128	2048	2048	0.1008	0.0317	0.1391	1.380
q_proj	512	2048	2048	0.1955	0.0910	0.4518	2.311
q_proj	2048	2048	2048	0.8650	0.2799	1.8723	2.165
k_proj	128	512	2048	0.0997	0.0227	0.1102	1.105
k_proj	512	512	2048	0.1051	0.0307	0.1453	1.382
k_proj	2048	512	2048	0.2198	0.0935	0.4577	2.082
v_proj	128	512	2048	0.0898	0.0280	0.1148	1.278
v_proj	512	512	2048	0.0950	0.0290	0.1390	1.463
v_proj	2048	512	2048	0.2172	0.0880	0.4467	2.057
o_proj	128	2048	2048	0.0930	0.0380	0.1586	1.705
o_proj	512	2048	2048	0.2004	0.0911	0.4477	2.234
o_proj	2048	2048	2048	0.8010	0.2782	1.8011	2.249
gate_proj	128	8192	2048	0.2171	0.1262	0.5134	2.365
gate_proj	512	8192	2048	0.8232	0.3135	1.7426	2.117
gate_proj	2048	8192	2048	3.5496	1.0637	7.4658	2.103
up_proj	128	8192	2048	0.2437	0.1271	0.5283	2.168
up_proj	512	8192	2048	0.8406	0.3235	1.8210	2.166
up_proj	2048	8192	2048	3.5433	1.0795	7.4318	2.097
down_proj	128	2048	8192	0.2261	0.1163	0.6336	2.802
down_proj	512	2048	8192	0.8665	0.3025	1.8284	2.110
down_proj	2048	2048	8192	3.3979	1.0720	8.9408	2.631

2. План и расписание (по `ORIGINAL_TASK.md`, 4 недели)

Ниже пункты плана из задания связаны с неделями работы.

Неделя 1. Triton: квантизация fp16→int4 с упаковкой и экономия памяти ~×4 (п. 1); matmul `X_bf16 @ W_4^T` (п. 2); сравнение с W16 на формах Llama-3.2-1B-Instruct и M ∈ {128, 512, 2048} (п. 3); тесты `pytest`; бенчмарк `exp01_bench_matmul.py`; черновик отчёта (`report/build.py`).

Неделя 2. Доработка ядер при необходимости; подготовка к квантизованному `nn.Linear` (п. 4).

Неделя 3. Квантизованный линейный слой на базе кернелей; загрузка весов Llama-3.2-1B-Instruct (п. 4).

Неделя 4. Замена линейных слоёв в модели; замеры скорости и перплексия на wikitext2 (п. 5); финальный отчёт.

Дополнительно по проекту: при необходимости CUDA-ядра и `torch.compile`.